
Title: ServiceNow to Azure DevOps Integration Using REST APIs

Hello, ServiceNow and Azure DevOps enthusiasts! In today's article, I will walk you through how to integrate **ServiceNow** with **Azure DevOps** using **REST APIs** to automatically create work items in Azure DevOps when incidents are raised in ServiceNow. This solution is especially helpful when traditional integration plugins aren't available or suitable for your environment, particularly if the plugins are paid and require licensing.

Use Case: Automatically Creating Work Items in Azure DevOps from ServiceNow Incidents

Problem:

When an incident or issue is raised in ServiceNow, it's crucial for development teams to track and resolve it. Without integration, the team would need to manually create corresponding work items in Azure DevOps, leading to duplication of effort, delays, and potential errors.

This integration solves that problem by automatically creating Azure DevOps work items when a new ServiceNow incident is created. The generated work item's task ID will be updated to the corresponding ServiceNow incident for traceability.

Flow of the Integration:

1. **Incident Creation in ServiceNow:**
    - A new incident or change request is created in **ServiceNow**, and relevant details like title, description, priority, and affected systems are filled in.
    - This incident needs to be tracked and resolved by the development team using **Azure DevOps**.
2. **Create Work Item in Azure DevOps via API:**
    - Once an incident is created in ServiceNow, an automated **REST API** call is triggered to create a corresponding work item in **Azure DevOps**.
    - The work item might include:
        - Incident description
        - Priority or severity level
        - Assigned developer or team
        - Associated tags or areas
    - The work item is linked back to the ServiceNow incident via the task ID for traceability.
3. **Update ServiceNow Incident with Work Item ID:**

- After the work item is successfully created in **Azure DevOps**, the **task ID** of the created work item is automatically updated in the corresponding **ServiceNow incident**.
- This ensures that the ServiceNow incident is linked to the Azure DevOps work item and provides traceability between the two systems.

How to Implement This Integration:

Let's walk through the integration process using **REST APIs**, starting with **ServiceNow** and **Azure DevOps**.

Step 1: Set Up REST Message in ServiceNow

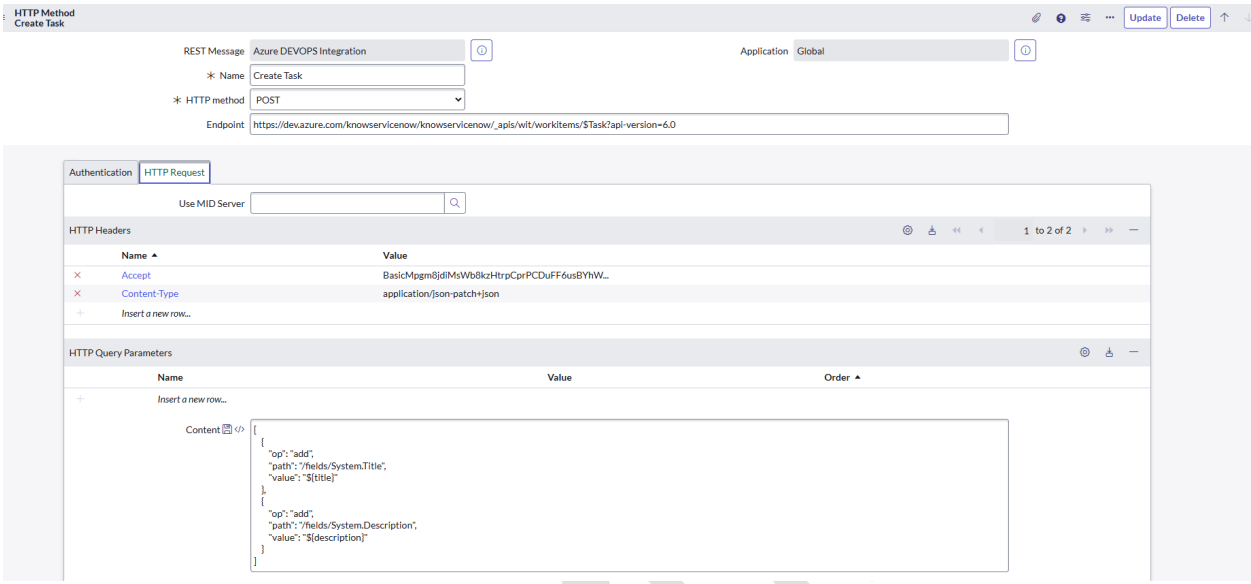
1. **Generate Personal Access Token (PAT) in Azure DevOps:**
   - First, generate a **Personal Access Token (PAT)** in **Azure DevOps** for authentication in the REST API calls.
2. **Create Outbound REST Message in ServiceNow:**
   - In ServiceNow, navigate to **System Web Services > Outbound > REST Message** and create a new REST message for the Azure DevOps work item integration.
   - Set the HTTP method to **POST** (for creating new work items) and configure the endpoint to match Azure DevOps' API.

Example Endpoint:

https://dev.azure.com/{organization}/{project}/_apis/wit/workitems/${workItemType}?api-version=6.0

Step 2: Define the HTTP Headers and Request Body

1. **HTTP Headers:**
   - **Content-Type:** application/json-patch+json
   - **Accept:** Include the Base64-encoded PAT for authentication.



2. **Request Body:**
    - The request body will map **ServiceNow** incident fields to **Azure DevOps** work item fields, such as:
        - Incident Title → Work Item Title
        - Incident Description → Work Item Description
        - Incident Priority → Work Item Priority

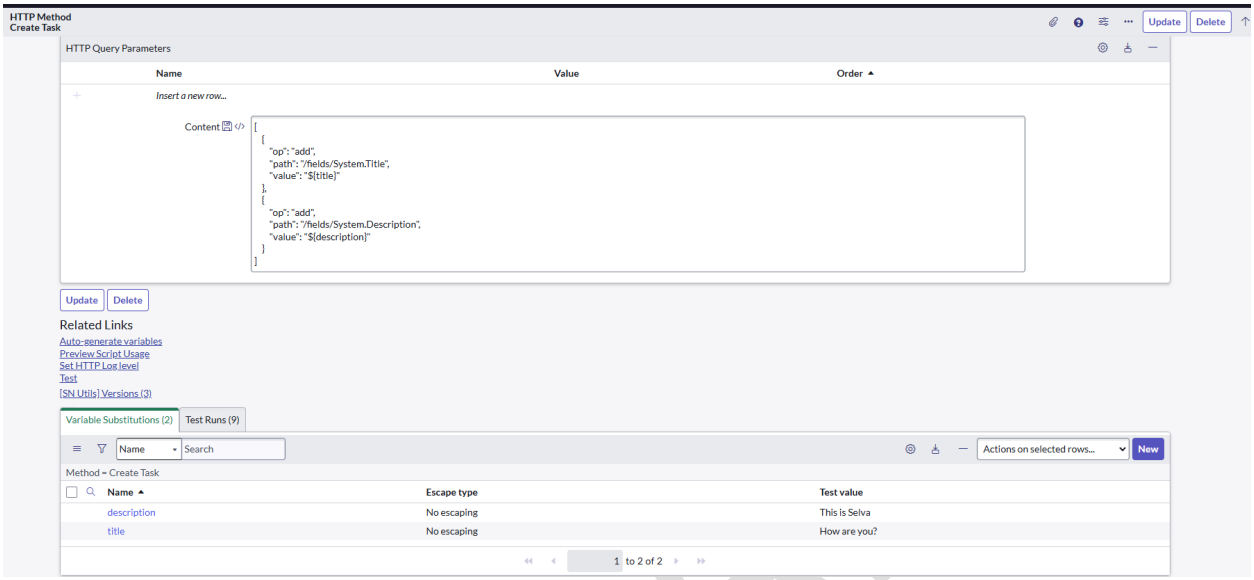
Example Body:

```
{
 "op": "add",
 "path": "/fields/System.Title",
 "value": "Incident: {incident_title}"
},
{
 "op": "add",
 "path": "/fields/System.Description",
 "value": "Description: {incident_description}"
},
{
 "op": "add",
```

"path": "/fields/System.Priority",

"value": "{incident_priority}"

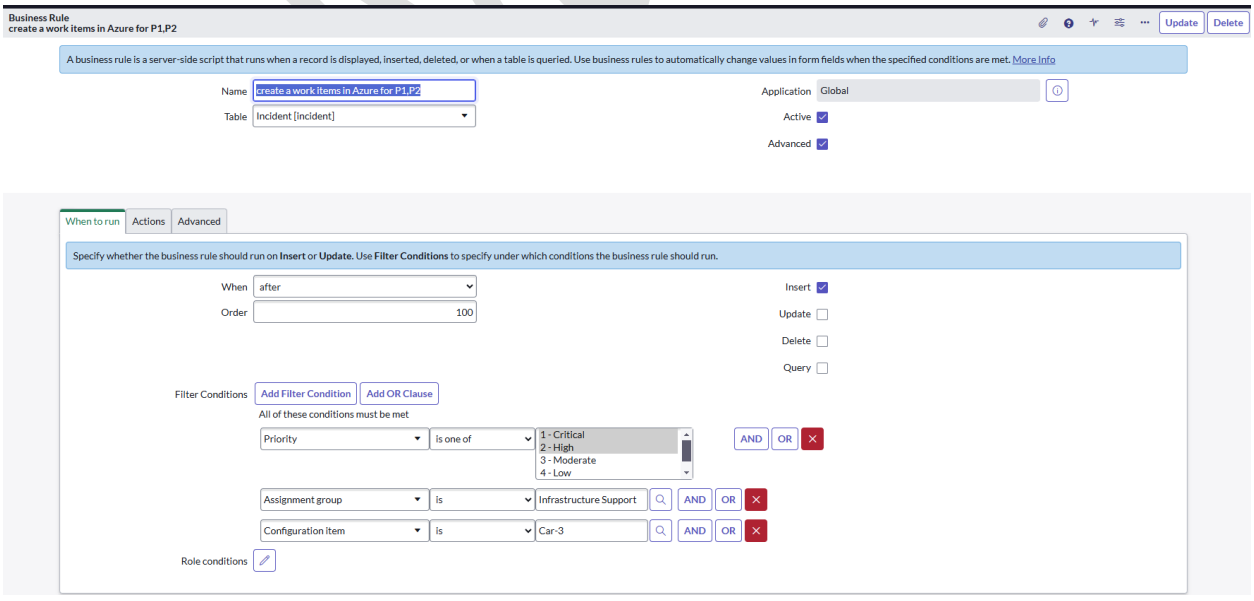
}



Step 3: Trigger the REST Message from ServiceNow

To trigger this integration when an incident is created or updated, you can use a **Business Rule** or **Script Include**.

Example Script (for triggering from a Business Rule) which I have used for my use case:



(function executeRule(current, previous) {

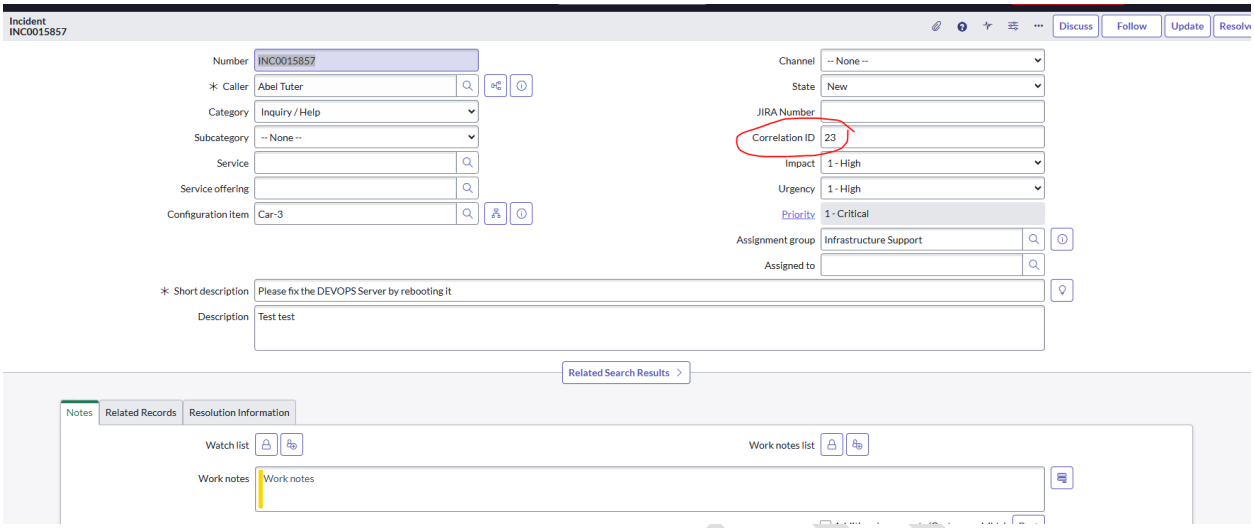
try {

```
    // Prepare REST message for Azure DevOps task creation
    var r = new sn_ws.RESTMessageV2('Azure DEVOPS Integration', 'Create Task');

    // Set necessary parameters for task creation
    r.setStringParameterNoEscape('description', current.description);
    r.setStringParameterNoEscape('title', current.short_description);

    // Execute the REST message and get the response
    var response = r.execute();
    var responseBody = response.getBody();
    var httpStatus = response.getStatusCode();

    // Check the response and handle accordingly
    if (httpStatus == 200) {
      var responseObj = JSON.parse(responseBody);
      current.correlation_id = responseObj.id;  // Update incident with Work Item ID
      current.update();
      gs.info('Task created successfully in Azure DevOps. Task ID: ' + responseObj.id);
    } else {
      gs.error('Failed to create task in Azure DevOps. Status Code: ' + httpStatus);
    }
  } catch (ex) {
    gs.error('Error occurred: ' + ex.message);
  }
})(current, previous);
```

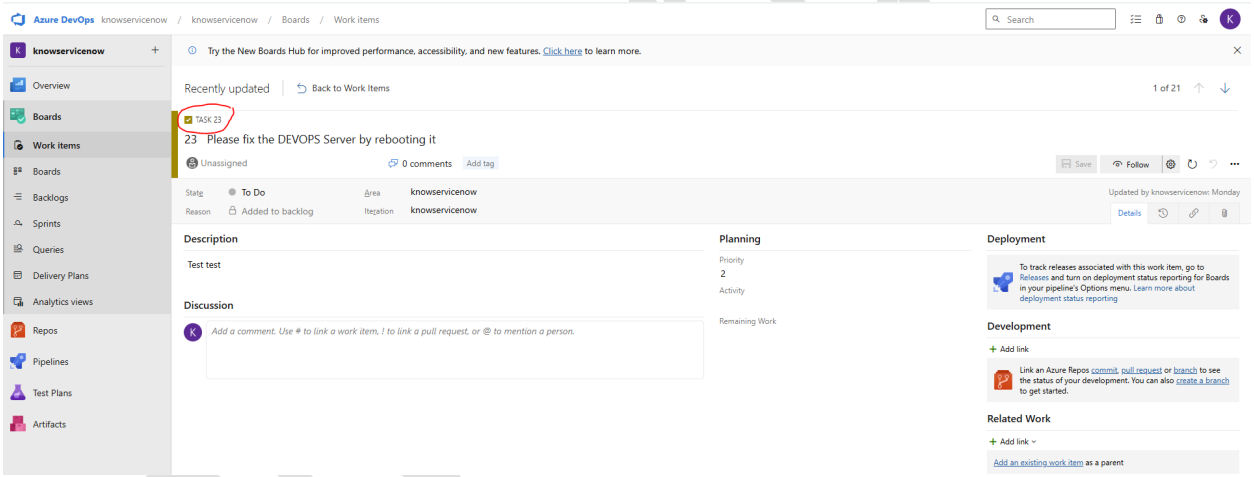
Step 4: Test the Integration

Once everything is set up, test the integration:

- Create an incident in **ServiceNow** and verify that a corresponding work item is automatically created in **Azure DevOps**.



- Check that the **task ID** of the created work item is updated in the **ServiceNow incident**.



Alternative Integration Option: Microsoft Azure DevOps Integration for Agile Development Plugin

While the method above achieves the integration via **REST APIs**, there is a more direct way to achieve the same result using the **Microsoft Azure DevOps Integration for Agile Development plugin**.

This plugin provides out-of-the-box integration between **ServiceNow** and **Azure DevOps**, including seamless work item creation, status synchronization, and more. However, the plugin comes with licensing fees and is **paid**, so it's ideal for organizations looking for a ready-made solution with minimal customization.

On the other hand, if you're aiming for a cost-effective solution and more control over the process, the **REST API** approach offers flexibility and customizability without the need for additional licenses.

Conclusion:

To summarize:

- I set up a **REST message** in **ServiceNow** to interact with **Azure DevOps** using **REST APIs**.
- Authentication, headers, and request body were configured to map **ServiceNow** incident details to **Azure DevOps** work items.
- The integration was triggered using a **Business Rule**, allowing the automated creation of work items in **Azure DevOps** based on **ServiceNow** incidents.
- The **task ID** from the created work item was then updated in the **ServiceNow incident**.

If you're looking for an out-of-the-box solution, the **Microsoft Azure DevOps Integration for Agile Development plugin** is an alternative, although it does come at a cost due to licensing requirements.

This method provides flexibility for integrating **ServiceNow** with **Azure DevOps** without relying on built-in plugins, making it a great solution for organizations that need a custom integration.

Thank you for reading! If you have any questions or need further assistance, feel free to leave a comment below. I'd love to help!

Also, I have uploaded a video- Learn To Integrate ServiceNow With Azure Devops Using Rest API in 20 mins!! on our YouTube channel explaining this process in detail. Please **like**, **share**, and **subscribe**, and don't forget to **leave your feedback**—I appreciate it! If you found this article helpful, please mark it as **useful**.

Useful Links:

- Azure DevOps REST API Documentation
